Technical Architecture

Ingestion is pluggable, extraction is LLM-driven and multi-label, and every table carries `tenant_id` enforced by Row-Level Security.

```
flowchart TD
  subgraph Sources["Data Sources"]
    EXC["Excel Upload<br/>Sprinklr/Brandwatch export"]
    API["Sprinklr / Brandwatch API<br/>(future)"]
    LAKE["Data Lake<br/>(future)"]
  end

  subgraph Ingestion["Ingestion Layer"]
    ADP["Adapter Interface<br/>pluggable per source"]
  end

  subgraph DB["Supabase Postgres — multi-tenant, RLS"]
    CONV["conversations<br/>+ raw_metadata"]
    TAX["Taxonomies:<br/>business / attribute / signal"]

TAGS["conversation_attribute_tags<br/>conversation_signal_tags<br/>conversation_brands"]
    BRANDS["brands / brand_aliases"]
  end

  subgraph Extract["Extraction Pipeline"]
    LLM["Gemini API<br/>multi-label tagging:<br/>attributes + signals + brand ID"]
  end

  subgraph Agg["Aggregation Layer"]
    ROLLUP["Brand x Attribute x Time<br/>scores"]
  end

  subgraph App["Application Layer"]
    DASH["Streamlit Dashboard"]
    AUTH["Supabase Auth<br/>RBAC: admin/analyst/viewer"]
  end

  EXC --> ADP
  API --> ADP
  LAKE --> ADP
  ADP --> CONV
  CONV --> LLM
  TAX --> LLM
```

```
BRANDS --> LLM
LLM --> TAGS
TAGS --> ROLLUP
CONV --> ROLLUP
ROLLUP --> DASH
AUTH --> DASH
DASH --> DB
```

Walkthrough

1. **Sources → Ingestion Layer** — today, only Excel Upload is implemented; the Adapter Interface is designed so Sprinklr/Brandwatch API adapters and a future data-lake adapter can be added without touching anything downstream.
2. **Ingestion → Database** — raw rows land in `conversations`, with the full untouched Sprinklr row preserved in `raw_metadata` (jsonb) so no field is ever lost even if not yet modeled as a column.
3. **Database + Taxonomies → Extraction Pipeline** — each conversation is sent to Gemini along with the taxonomy definitions and known brand aliases; the model returns attribute tags, signal tags, and brand matches in one structured pass.
4. **Extraction → Tags tables** — results are written to the three junction tables (`conversation_attribute_tags`, `conversation_signal_tags`, `conversation_brands`), preserving the multi-label, no-double-counting design.
5. **Tags + Conversations → Aggregation** — a rollup job computes brand × attribute × time-period scores from the tagged data.
6. **Aggregation → Dashboard** — Streamlit reads only the aggregated scores (not raw conversations) for fast rendering; Supabase Auth enforces per-tenant, per-role access on every query.